

PROGRAM 10:-

AIM

To implement the A* algorithm to find the shortest path between a starting node and a goal node using the actual cost and heuristic cost.

ALGORITHM

1. Start the program.
2. Define the graph with nodes and their connecting edges.
3. Define the heuristic value for each node.
4. Set the starting node and goal node.
5. Add the starting node to the open list.
6. Calculate the value of $f(n) = g(n) + h(n)$.
7. Select the node with the lowest $f(n)$ value.
8. Expand the selected node and find its neighbouring nodes.
9. Calculate the cost for each neighbour.
10. Update the path and cost if a better path is found.
11. Repeat the process until the goal node is reached.
12. Display the shortest path and its total cost.
13. Stop the program.

PROGRAM

```
import heapq

def a_star(graph, heuristic, start, goal):
    open_list = []
    heapq.heappush(open_list, (0, start))
    g_cost = {start: 0}
    parent = {start: None}
    while open_list:
        f_cost, current = heapq.heappop(open_list)
        if current == goal:
            path = []
```

```

while current is not None:
    path.append(current)
    current = parent[current]
path.reverse()
return path, g_cost[goal]
for neighbour, cost in graph[current]:
    new_cost = g_cost[current] + cost
    if neighbour not in g_cost or new_cost < g_cost[neighbour]:
        g_cost[neighbour] = new_cost
        f = new_cost + heuristic[neighbour]
        parent[neighbour] = current
        heapq.heappush(open_list, (f, neighbour))
return None, None
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 3)],
    'D': [('G', 3)],
    'E': [('G', 1)],
    'F': [('G', 2)],
    'G': []
}
heuristic = {
    'A': 6,
    'B': 5,
    'C': 4,
    'D': 3,
    'E': 1,
    'F': 2,

```

```
'G': 0
}
path, cost = a_star(graph, heuristic, 'A', 'G')
print("Shortest Path:", " -> ".join(path))
print("Total Cost:", cost)
```

OUTPUT

```
Shortest Path: A -> B -> D -> G
Total Cost: 6
```

RESULT

The A* algorithm was successfully implemented, and the shortest path from the starting node A to the goal node G was obtained with a minimum cost of 6.